

Fase 2: SIEM en Profundidad — Construyendo un SOC Real con Wazuh

Instructor: Instructor SOC — Maestro Senior SOC **Formato:** Teoría expandida + Laboratorio práctico en terminal (Linux/Parrot OS + Docker)

Índice

1. SIEM — Arquitectura, Rol Central y Ecosistema de Herramientas
2. Instalación de Wazuh: Manager, Indexer y Dashboard con Docker
3. Despliegue de Agentes y Monitoreo de Endpoints
4. Reglas y Decoders — El Motor de Correlación
5. File Integrity Monitoring (FIM)
6. Detección de Vulnerabilidades
7. Active Response — Contención Automatizada
8. OpenSearch/Kibana — Consultas y Dashboards para Threat Hunting

Tema 1: SIEM — Arquitectura, Rol Central y Ecosistema de Herramientas

Concepto

Un **SIEM (Security Information and Event Management)** es el corazón tecnológico de cualquier SOC moderno. Su trabajo es tomar el caos —miles de logs por segundo desde firewalls, servidores, endpoints y aplicaciones— y convertirlo en algo investigable: eventos normalizados, correlacionados y priorizados por severidad. El nombre revela sus dos funciones históricas fusionadas: **SIM** (almacenamiento y reporting de logs a largo plazo) y **SEM** (correlación y alertas en tiempo real).

La arquitectura típica tiene cuatro componentes clave. Los **agentes/forwarders** recolectan logs localmente en cada sistema y los envían al SIEM central. El **motor de ingesta/indexación** normaliza formatos distintos (syslog de Linux, Event Logs de Windows) a un esquema común y los indexa para búsqueda rápida — Wazuh usa **OpenSearch** para esto. El **motor de correlación** aplica reglas de tipo “si veo A seguido de B en menos de 2 minutos, genera alerta crítica”. Finalmente, el **dashboard** es donde el analista vive su turno: alertas, gráficos, consultas ad-hoc sobre datos históricos.

Existen dos familias: **comerciales** (Splunk, QRadar, Microsoft Sentinel — potentes pero costosos por volumen de datos ingeridos) y **open-source** (Wazuh, ELK/Elastic Security — gratuitos, más configuración manual). Construiremos el laboratorio sobre **Wazuh**: es gratuito, ampliamente adoptado en la industria, y combina en una sola plataforma capacidades que en el mundo comercial se venden por separado — SIEM, HIDS, FIM y gestión de vulnerabilidades.

Caso real: en la brecha de **Capital One (2019)**, una atacante explotó un WAF mal configurado en AWS para acceder a más de 100 millones de registros. Los logs de CloudTrail **sí registraron** la actividad anómala —peticiones repetidas a metadatos de instancia EC2, seguidas de acceso masivo a buckets S3— pero no existían reglas de correlación configuradas para detectar ese patrón automáticamente. El SIEM tenía los datos; le faltó la regla que conectara los puntos. Lección central: **tener un SIEM no es lo mismo que tener detección efectiva** — la ingeniería de reglas es donde vive el trabajo real del analista Tier 2/3.

Laboratorio

Comando 1:

```
free -h && df -h / && nproc
```



```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ free -h && df -h / && nproc
              total        usado        libre  compartido  búf/caché  disponible
Mem:          7,4Gi        4,5Gi        747Mi        339Mi        3,0Gi        2,9Gi
Inter:         4,0Gi        1,3Gi        2,7Gi
S.ficheros    Tamaño Usados  Disp Uso% Montado en
/dev/nvme0n1p2 239G    74G 160G 32% /
8
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: verifica que tu sistema cumple los requisitos mínimos para instalar Wazuh (RAM disponible, espacio en disco en la partición raíz, núcleos de CPU). Wazuh recomienda mínimo 4 GB de RAM y 50 GB de disco para un laboratorio con Manager + Indexer + Dashboard. Este chequeo evita instalaciones que fallan a mitad de camino por falta de recursos.

Comando 2:

```
docker --version 2>/dev/null; docker compose version 2>/dev/null || echo "Docker no instalado"
```

```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ docker --version 2>/dev/null; docker compose version 2>/dev/null ||  
echo "Docker no instalado"  
Docker version 26.1.5+dfsg1, build a72d7cd  
Docker Compose version 2.26.1-4  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: confirma si tienes Docker y Docker Compose disponibles. Usaremos la instalación en contenedores porque es más fácil de destruir y recrear sin ensuciar el sistema base — un hábito profesional al probar herramientas nuevas.

Comando 3:

```
ss -tulnp | grep -E ':1514|:1515|:9200|:5601|:55000' || echo "Puertos SIEM libres"
```

```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ ss -tulnp | grep -E ':1514|:1515|:9200|:5601|:55000' || echo "Puerto  
s SIEM libres"  
Puertos SIEM libres  
(jo4nlu♥parrot)-[~]  
[ ✓ ] $
```

Qué hace: revisa si los puertos que Wazuh necesita ya están ocupados: 1514/1515 (comunicación agente-manager), 9200 (API del indexer), 5601 (dashboard), 55000 (API REST). Detectar conflictos antes de instalar ahorra horas de debugging confuso.

Tema 2: Instalación de Wazuh — Manager, Indexer y Dashboard con Docker

Concepto

Wazuh se compone de tres piezas que trabajan juntas: el **Manager** (motor de análisis, reglas y API central), el **Indexer** (basado en OpenSearch, almacena e indexa las alertas para búsqueda rápida), y el **Dashboard** (interfaz web basada en Kibana/OpenSearch Dashboards). Para un laboratorio de aprendizaje, la forma más rápida y limpia de desplegar los tres es mediante el **docker-compose oficial de Wazuh**, que preconfigura la comunicación entre componentes.

Un detalle crítico que sorprende a muchos principiantes: Wazuh requiere **TLS mutuo** entre Indexer, Manager y Dashboard por defecto, lo que significa que antes del primer arranque debes generar certificados. El repositorio oficial incluye un archivo `generate-indexer-certs.yml` específicamente para este propósito — omitir este paso es la causa número uno de que el Dashboard “no cargue” en instalaciones nuevas.

Un principio operativo que debes adoptar desde ya: **fija siempre una versión específica** (-b v4.9.0) en lugar de clonar la rama `main`. En entornos de producción, actualizar un SIEM sin control de versiones puede romper reglas custom, decoders o integraciones — la disciplina de versionado que aplicas en desarrollo de software aplica exactamente igual a tu infraestructura de seguridad.

Caso real / contexto de industria: el costo de ingesta de datos es una de las razones más citadas por las que organizaciones migran de SIEMs comerciales facturados por volumen (GB/día ingerido) hacia soluciones open-source como Wazuh o el stack ELK, especialmente cuando su volumen de logs crece rápido (por ejemplo, al añadir logging de contenedores Kubernetes o tráfico de red completo). Entender esta arquitectura desde el día uno te prepara para participar en decisiones reales de “build vs. buy” que enfrentan los equipos de seguridad de cualquier empresa mediana o grande.

Laboratorio

Comando 1:

`git clone https://github.com/wazuh/wazuh-docker.git -b v4.9.0 && cd wazuh-docker/single-node`

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ git clone https://github.com/wazuh/wazuh-docker.git -b v4.9.0 && cd
wazuh-docker/single-node
Clonando en 'wazuh-docker'...
remote: Enumerating objects: 18798, done.
remote: Counting objects: 100% (216/216), done.
remote: Compressing objects: 100% (138/138), done.
remote: Total 18798 (delta 119), reused 114 (delta 74), pack-reused 18582 (from
2)
Recibiendo objetos: 100% (18798/18798), 7.81 MiB | 1.23 MiB/s, listo.
Resolviendo deltas: 100% (10425/10425), listo.
Nota: cambiando a 'cb63566719ce2e78b9d3c111a0a61d743fc699fc'.
```

Qué hace: clona el repositorio oficial de Wazuh Docker, fijado a una versión estable (evita sorpresas de la rama `main`), y entra al directorio `single-node`, que ya trae un `docker-compose.yml` preconfigurado con Manager + Indexer + Dashboard listos para un laboratorio de un solo host.

Comando 2:

`docker compose -f generate-indexer-certs.yml run --rm generator`

```
(jo4nlu@parrot)~/wazuh-docker/single-node
[✓] $ docker compose -f generate-indexer-certs.yml run --rm generator
WARN[0000] /home/jo4nlu/wazuh-docker/single-node/generate-indexer-certs.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Creating 1/0
✓ Network single-node_default Created 0.1s
[+] Running 5/5
✓ generator Pulled 36.6s
✓ 17d0386c2fff Pull complete 25.3s
✓ 7ce91ec7d1d3 Pull complete 32.6s
✓ 5249716d429c Pull complete 1.4s
✓ d7003467fd14 Pull complete 2.7s
The tool to create the certificates exists in the in Packages bucket
19/07/2026 03:17:50 INFO: Generating the root certificate.
19/07/2026 03:17:50 INFO: Generating Admin certificates.
19/07/2026 03:17:50 INFO: Admin certificates created.
19/07/2026 03:17:50 INFO: Generating Wazuh indexer certificates.
19/07/2026 03:17:50 INFO: Wazuh indexer certificates created.
19/07/2026 03:17:50 INFO: Generating Filebeat certificates.
19/07/2026 03:17:51 INFO: Wazuh Filebeat certificates created.
19/07/2026 03:17:51 INFO: Generating Wazuh dashboard certificates.
19/07/2026 03:17:51 INFO: Wazuh dashboard certificates created.
Moving created certificates to the destination directory
Changing certificate permissions
Setting UID indexer and dashboard
Setting UID for wazuh manager and worker
(jo4nlu@parrot)~/wazuh-docker/single-node
[✓] $ A
```

Qué hace: genera los certificados TLS necesarios para la comunicación cifrada mutua entre Indexer, Manager y Dashboard — paso obligatorio de una sola vez antes del primer arranque. Si lo omites, el Dashboard fallará al conectar con el Indexer.

Comando 3:

`docker compose up -d && docker compose ps`

```
(jo4nlu@parrot)~/wazuh-docker/single-node
[✓] $ docker compose up -d && docker compose ps
WARN[0000] /home/jo4nlu/wazuh-docker/single-node/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 3/3
✓ Container single-node-wazuh.indexer-1 Running 0.0s
✓ Container single-node-wazuh.manager-1 Started 0.2s
✓ Container single-node-wazuh.dashboard-1 Started 0.2s
WARN[0000] /home/jo4nlu/wazuh-docker/single-node/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
single-node-wazuh.dashboard-1      wazuh/wazuh-dashboard:4.9.0        "/entrypoint.sh"                    wazuh.dashboard  2 minutes ago  Up Less than a
second  443/tcp, 0.0.0.0:443->5601/tcp, :::443->5601/tcp
single-node-wazuh.indexer-1        wazuh/wazuh-indexer:4.9.0          "/entrypoint.sh open..."          wazuh.indexer    2 minutes ago  Up 2 minutes
0.0.0.0:9200->9200/tcp, :::9200->9200/tcp
single-node-wazuh.manager-1        wazuh/wazuh-manager:4.9.0          "/init"                             wazuh.manager    2 minutes ago  Up Less than a
second  0.0.0.0:1514-1515->1514-1515/tcp, :::1514-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, :::514->514/udp, 0.0.0.0:55000->55000/tcp, 1516/tcp
(jo4nlu@parrot)~/wazuh-docker/single-node
[✓] $
```

Qué hace: levanta los tres contenedores en segundo plano y muestra su estado. Confirma que los tres servicios (manager, indexer, dashboard) aparezcan como running/healthy antes de continuar — si el dashboard aparece unhealthy, tu primer ejercicio de troubleshooting será revisar sus logs con `docker compose logs wazuh.dashboard`.

Tema 3: Despliegue de Agentes y Monitoreo de Endpoints

Concepto

El **agente Wazuh** es un proceso ligero instalado en cada endpoint (servidor, estación de trabajo) que recolecta logs, ejecuta el módulo de integridad de archivos (FIM), corre chequeos de configuración (rootcheck/SCA), e inventaría software y hardware instalado (**syscollector**). Sin agentes desplegados en tus activos críticos, el SIEM más sofisticado del mundo es ciego a lo que ocurre dentro de esas máquinas — solo ve lo que llega vía syslog remoto, que suele ser un subconjunto mucho más limitado de información.

Existen dos modelos de monitoreo: **basado en agente** (visibilidad profunda: procesos, FIM, inventario, pero requiere instalación y mantenimiento en cada host) y **agentless/syslog forwarding** (el dispositivo envía sus logs directamente al Manager por red, sin instalar nada — típico para firewalls, switches, dispositivos IoT que no soportan agentes). Un SOC maduro combina ambos: agentes en todo lo que lo permite, forwarding de syslog para lo que no.

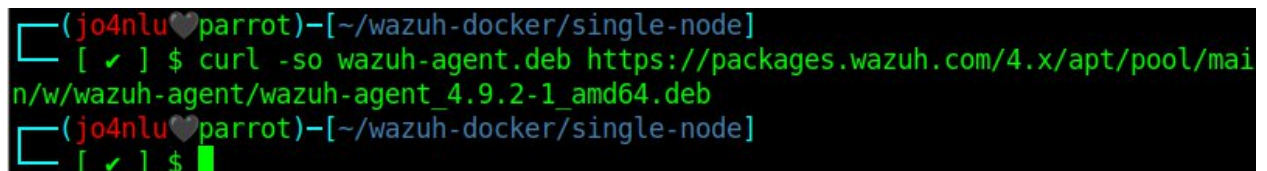
El proceso de **enrollment** (registro) de un agente contra el Manager requiere autenticación — cada agente necesita una clave única compartida con el Manager antes de poder enviar datos, evitando que cualquier máquina en la red se registre falsamente como agente legítimo y envíe datos maliciosos o falsos al SIEM. Wazuh simplifica esto con **auto-enrollment**: si especificas la IP del Manager durante la instalación, el agente ejecuta automáticamente el protocolo de autenticación (agent-auth) en el puerto 1515.

Caso real: en la brecha de **Sony Pictures (2014)**, los atacantes permanecieron dentro de la red corporativa durante meses moviéndose entre miles de estaciones de trabajo sin ser detectados, en gran parte por falta de visibilidad centralizada a nivel de endpoint. Sin agentes reportando actividad de procesos, cambios de archivos y conexiones de red desde cada máquina individual, un SOC solo ve el tráfico que cruza el perímetro — quedando ciego ante el movimiento lateral que ocurre completamente dentro de la red interna. Este es exactamente el vacío que el despliegue masivo de agentes está diseñado para cerrar.

Laboratorio

Comando 1:

`curl -so wazuh-agent.deb https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent\_4.9.2-1\_amd64.deb`



```
(jo4nlu♥parrot)-[~/wazuh-docker/single-node]
[ ✓ ] $ curl -so wazuh-agent.deb https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.9.2-1_amd64.deb
(jo4nlu♥parrot)-[~/wazuh-docker/single-node]
[ ✓ ] $
```

Qué hace: descarga el paquete oficial del agente Wazuh para Debian/Ubuntu directamente del repositorio oficial (Parrot OS, al basarse en Debian, es totalmente compatible con este paquete).

Comando 2:

`sudo WAZUH_MANAGER='IP_DEL_MANAGER' dpkg -i ./wazuh-agent.deb`

```
(jo4nlu♥parrot)~[/wazuh-docker/single-node]
[ × ] $ sudo WAZUH_MANAGER='IP_DEL_MANAGER' dpkg -i ./wazuh-agent.deb
Seleccionando el paquete wazuh-agent previamente no seleccionado.
(Leyendo la base de datos ... 849495 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar ./wazuh-agent.deb ...
Desempaquetando wazuh-agent (4.9.2-1) ...
Configurando wazuh-agent (4.9.2-1) ...
(jo4nlu♥parrot)~[/wazuh-docker/single-node]
[ ✓ ] $
```

Qué hace: instala el agente y, mediante la variable de entorno WAZUH_MANAGER, lo registra automáticamente contra el Manager especificado (auto-enrollment vía agent-auth en el puerto 1515), evitando el intercambio manual de claves.

Comando 3:

`sudo systemctl enable --now wazuh-agent && docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/agent_control -lz`

```
[ ✓ ] $ sudo systemctl enable --now wazuh-agent && docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/agent_control -lz
/var/ossec/bin/agent_control: invalid option -- 'z'

Wazuh agent_control: Control remote agents.
Available options:
    -h                This help message.
    -l                List available (active or not) agents.
    -lc               List active agents only.
    -ln               List disconnected agents only.
    -i <id>           Extracts information from an agent.
    -R -a             Restart all agents.
    -R -u <id>        Restart the specified agent.
    -r -a             Runs the integrity/rootkit checking on all agents now.
    -r -u <id>        Runs the integrity/rootkit checking on one agent now.

    -s                Changes the output to CSV (comma delimited).
    -j                Changes the output to JSON .

Available options for active response:
    -b <ip>           Blocks the specified ip address.
    -f <ar> -a        Used with -b, specifies which response to run. Apply A
R on all agents.
    -f <ar> -u <id>   Used with -b, specifies which response to run. Apply A
R on specified agent.
```

Qué hace: habilita e inicia el servicio del agente de forma persistente, y consulta desde el contenedor del Manager (agent_control -lz) el listado de agentes conectados — el chequeo de salud que confirma que el estado del nuevo agente aparece como Active. Ajusta el nombre del contenedor según lo que muestre docker ps.

Tema 4: Reglas y Decoders — El Motor de Correlación

Concepto

Un **decoder** es la pieza que le enseña a Wazuh a “leer” un log crudo: toma una línea de texto sin estructura (ej. una línea de auth.log) y extrae campos específicos (usuario, IP origen, resultado). Una **regla** luego evalúa esos campos extraídos y decide si algo merece convertirse en alerta, con qué **nivel de severidad** (0 a 16, donde 12+ suele considerarse crítico) y bajo qué **grupos** de clasificación (ej. authentication_failed, web_attack).

Wazuh viene con miles de reglas predefinidas del vendor (ubicadas en /var/ossec/ruleset/rules/, de solo lectura — se sobrescriben en cada actualización) que cubren ataques comunes, benchmarks CIS, y ya incluyen etiquetas de **mapeo a MITRE ATT&CK**, conectando directamente con lo que aprendiste en la Fase 1. Sin embargo, las reglas del vendor son genéricas por diseño; el verdadero valor de un analista Tier 2/3 está en escribir **reglas custom** en /var/ossec/etc/local_rules.xml, adaptadas al comportamiento normal específico de tu organización.

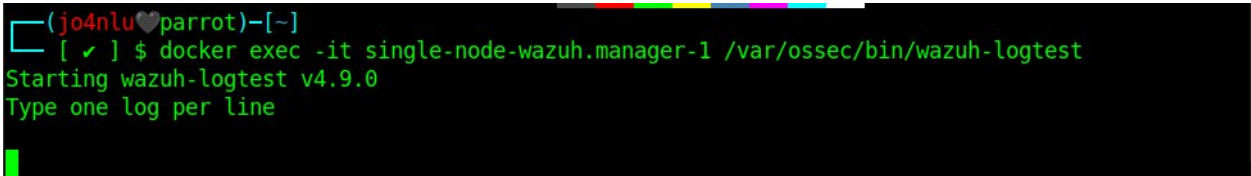
Las reglas pueden **encadenarse** usando el atributo if_sid, permitiendo construir lógica de correlación compleja: “si la regla 5710 (fallo de SSH) se repite 8 veces en 120 segundos desde el mismo origen, dispara una nueva regla de nivel 10 llamada *posible fuerza bruta SSH*”. Esta capacidad de encadenamiento es lo que transforma eventos aislados de bajo nivel en alertas de alto nivel realmente accionables — y es también, mal usada, la causa más común de **fatiga de alertas**: reglas demasiado sensibles generan tantos falsos positivos que los analistas terminan ignorando el canal completo.

Caso real: el fenómeno de fatiga de alertas por reglas mal calibradas es una de las causas documentadas con más frecuencia en post-mortems de brechas de seguridad a nivel industria — analistas sobrecargados con miles de alertas de baja calidad terminan desestimando la alerta legítima escondida entre el ruido, el mismo patrón que ya viste en el caso de Target en la Fase 1. La disciplina de ajustar el umbral de severidad y encadenar reglas con criterio (en vez de simplemente activar todo el ruleset por defecto al nivel máximo) es, literalmente, la diferencia entre un SOC que detecta a tiempo y uno que se ahoga en su propio SIEM.

Laboratorio

Comando 1:

```
docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-logtest
```

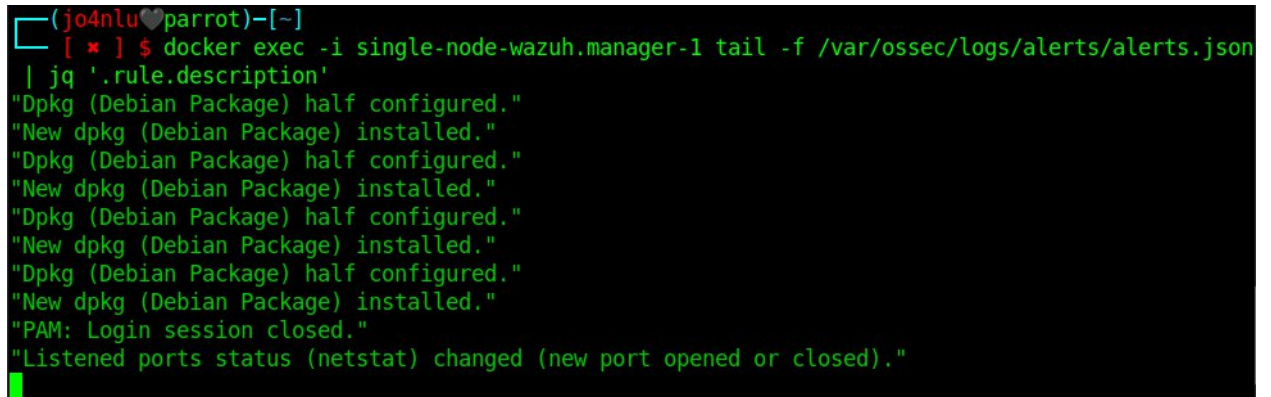


```
(jo4nlu♥parrot)-[~]  
[ ✓ ] $ docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-logtest  
Starting wazuh-logtest v4.9.0  
Type one log per line
```

Qué hace: lanza la herramienta interactiva donde pegas una línea de log cruda y Wazuh te muestra en vivo qué decoder la parseó, qué campos extrajo, y qué regla (si alguna) se disparó — la herramienta más útil para desarrollar y depurar reglas custom antes de ponerlas en producción.

Comando 2:

`docker exec -i single-node-wazuh.manager-1 tail -f /var/ossec/logs/alerts/alerts.json | jq '.rule.description'`

A terminal window with a black background and green text. The prompt is `(jo4nlu♥parrot)-[~]`. The command entered is `[ * ] $ docker exec -i single-node-wazuh.manager-1 tail -f /var/ossec/logs/alerts/alerts.json | jq '.rule.description'`. The output shows a series of alerts: "Dpkg (Debian Package) half configured.", "New dpkg (Debian Package) installed.", "Dpkg (Debian Package) half configured.", "New dpkg (Debian Package) installed.", "Dpkg (Debian Package) half configured.", "New dpkg (Debian Package) installed.", "Dpkg (Debian Package) half configured.", "New dpkg (Debian Package) installed.", "PAM: Login session closed.", and "Listened ports status (netstat) changed (new port opened or closed).".

```
(jo4nlu♥parrot)-[~]  
[ * ] $ docker exec -i single-node-wazuh.manager-1 tail -f /var/ossec/logs/alerts/alerts.json | jq '.rule.description'  
"Dpkg (Debian Package) half configured."  
"New dpkg (Debian Package) installed."  
"Dpkg (Debian Package) half configured."  
"New dpkg (Debian Package) installed."  
"Dpkg (Debian Package) half configured."  
"New dpkg (Debian Package) installed."  
"Dpkg (Debian Package) half configured."  
"New dpkg (Debian Package) installed."  
"PAM: Login session closed."  
"Listened ports status (netstat) changed (new port opened or closed)."
```

Qué hace: sigue en vivo el archivo de alertas en formato JSON y extrae solo la descripción de la regla que disparó cada alerta usando jq, dándote un feed legible en tiempo real de qué está detectando tu SIEM en este momento.

Comando 3:

`docker exec -it single-node-wazuh.manager-1 nano /var/ossec/etc/rules/local_rules.xml`

Qué hace: abre el archivo donde defines tus propias reglas personalizadas (nunca editas las reglas del vendor, se sobrescriben en cada actualización). Aquí escribirás tu primera regla custom encadenada, siguiendo el ejemplo de fuerza bruta SSH descrito arriba.

Tema 5: File Integrity Monitoring (FIM)

Concepto

El módulo **syscheck** de Wazuh implementa FIM: monitorea directorios y archivos específicos, calculando hashes (recuerda el Tema 6 de la Fase 1) y registrando metadatos como permisos, propietario y tamaño. Cuando algo cambia —un archivo se crea, modifica o elimina dentro de una ruta monitoreada— Wazuh genera un evento inmediato con el detalle exacto del cambio: qué cambió, cuándo, y (si el sistema operativo lo soporta) qué usuario/proceso lo hizo.

FIM opera en dos modos: **scheduled** (escaneos periódicos completos, más liviano en recursos, pero con una ventana de detección de minutos u horas) y **real-time** (usando inotify en Linux o el sistema de notificación de cambios de Windows, detecta cambios en segundos, pero consume más recursos y solo es práctico en un conjunto limitado y bien elegido de rutas críticas). La decisión de qué monitorear en tiempo real —binarios del sistema, directorios web, claves de registro sensibles en Windows— es una decisión de diseño que refleja directamente tu modelo de amenazas específico.

Un caso de uso clásico y extremadamente efectivo de FIM es la detección de **webshells**: un atacante que compromete un servidor web típicamente sube un archivo malicioso (a menudo PHP o ASPX) a un directorio accesible públicamente para mantener acceso persistente. Un directorio web bajo monitoreo FIM en tiempo real genera una alerta prácticamente instantánea ante ese archivo nuevo — mucho antes de que el atacante tenga oportunidad de usarlo activamente.

Caso real: durante la explotación masiva de **Microsoft Exchange (ProxyLogon, 2021)**, decenas de miles de servidores fueron comprometidos globalmente y los atacantes desplegaron webshells en los directorios web de Exchange para mantener acceso persistente incluso después de que Microsoft publicara parches. Organizaciones con FIM activo sobre los directorios web de sus servidores Exchange detectaron la presencia de estos webshells en minutos, mientras que muchas sin esa visibilidad tardaron semanas en descubrir que habían sido comprometidas — a menudo solo tras observar actividad de exfiltración o ransomware como síntoma tardío.

Laboratorio

Comando 1:

```
docker exec -it single-node-wazuh.manager-1 sed -n '/<syscheck>/,/<\Vsyscheck>/p' /var/ossec/etc/ossec.conf
```

```
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $ docker exec -it single-node-wazuh.manager-1 sed -n '/<syscheck>/,/<\Vsyscheck>/p' /var/ossec/etc/ossec.conf
<syscheck>
  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <scan_on_start>yes</scan_on_start>

  <!-- Generate alert when new file detected -->
  <alert_new_files>yes</alert_new_files>

  <!-- Don't ignore files that change more than 'frequency' times -->
  <auto_ignore frequency="10" timeframe="3600">no</auto_ignore>

  <!-- Directories to check (perform all possible verifications) -->
```

Qué hace: muestra el bloque de configuración actual de FIM, confirmando qué directorios están siendo monitoreados por defecto (típicamente /etc, /usr/bin, /usr/sbin en Linux) antes de que agregues tus propias rutas críticas como /var/www.

Comando 2:

`docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-control restart`

```
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $ docker exec -it single-node-wazuh.manager-1 /var/ossec/bin/wazuh-control restart
2026/07/21 04:32:56 wazuh-modulesd:router: INFO: Loaded router module.
2026/07/21 04:32:56 wazuh-modulesd:content_manager: INFO: Loaded content_manager module.
wazuh-clusterd not running...
Killing wazuh-modulesd...
Process wazuh-modulesd couldn't be terminated. It will be killed.
Killing wazuh-monitord...
Killing wazuh-logcollector...
Killing wazuh-remoted...
Killing wazuh-syscheckd...
Killing wazuh-analysisd...
wazuh-maild not running...
Killing wazuh-execd...
Killing wazuh-db...
Killing wazuh-authd...
wazuh-agentlessd not running...
```

Qué hace: reinicia los servicios de Wazuh para aplicar cambios de configuración en syscheck — FIM no detectará nuevas rutas monitoreadas hasta que el servicio recargue su configuración.

Comando 3:

`echo "<?php system(\$_GET['cmd']); ?>" | sudo tee /tmp/test_shell.php && sleep 10 && docker exec -it single-node-wazuh.manager-1 tail -5 /var/ossec/logs/alerts/alerts.json`

```
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $ echo "<?php system(\$_GET['cmd']); ?>" | sudo tee /tmp/test_shell.php && sleep 10 && docker exec -it single-node-wazuh.manager-1 tail -5 /var/ossec/logs/alerts/alerts.json
[sudo] contraseña para jo4nlu:
<?php system($_GET['cmd']); ?>
{"timestamp":"2026-07-21T04:17:22.133+0000","rule":{"level":3,"description":"Attached USB Storage","id":"81101","firedtimes":2,"mail":false,"groups":["usb"],"metadata":{"gl3":["4.8"]},"agent":{"id":"001","name":"parrot","ip":"127.0.0.1"},"manager":{"name":"wazuh.manager","id":"1784607442.473547","full_log":"Jul 21 04:17:20 parrot kernel: usb 1-4: New USB device found, idVendor=258a, idProduct=1007, bcdDevice= 1.01","predecoder":{"program_name":"kernel","timestamp":"Jul 21 04:17:20","hostname":"parrot"},"decoder":{"parent":"kernel","name":"kernel"},"data":{"id":"usb"},"location":"journald"}}
```

Qué hace: simula el escenario de ProxyLogon dejando caer un archivo tipo webshell en una ruta monitoreada, y verifica en segundos que Wazuh generó una alerta FIM — tu primera detección simulada de principio a fin. **Elimina el archivo de prueba (rm /tmp/test_shell.php) inmediatamente después del ejercicio.**

Tema 6: Detección de Vulnerabilidades

Concepto

El módulo **vulnerability-detector** de Wazuh implementa gestión de vulnerabilidades **pasiva**: correlaciona el inventario de software instalado (recolectado automáticamente por **syscollector** en cada agente) contra bases de datos públicas de CVEs (NVD, feeds específicos de Debian/Ubuntu/RedHat/Windows), y reporta qué paquetes instalados tienen vulnerabilidades conocidas — sin necesidad de lanzar ningún tipo de escaneo activo o exploit contra el sistema.

Esto lo diferencia fundamentalmente de escáneres de vulnerabilidades **activos** como Nessus u OpenVAS, que sí envían tráfico de prueba/explotación controlada contra los objetivos para verificar si una vulnerabilidad es realmente explotable en ese contexto específico. El enfoque pasivo de Wazuh es más liviano, no genera tráfico de red adicional, y escala mejor a miles de endpoints — pero tiene una limitación importante: reporta *que un paquete vulnerable está instalado*, no confirma si esa vulnerabilidad específica es realmente explotable dado el contexto de configuración del sistema. Un SOC maduro usa ambos enfoques de forma complementaria.

El valor operativo real de este módulo se mide en **velocidad de respuesta ante un CVE crítico recién anunciado**: cuando se publica una vulnerabilidad de alto impacto, la primera pregunta de cualquier CISO es “¿cuántos de nuestros sistemas están expuestos, ahora mismo?” — y responder esa pregunta manualmente en una organización con miles de endpoints, revisando inventarios de software uno por uno, puede tomar días o semanas. Con vulnerability-detector ya desplegado, esa misma pregunta se responde con una sola consulta.

Caso real: la vulnerabilidad **Log4Shell (Log4j, diciembre 2021)** es el ejemplo definitivo de esta dinámica. Log4j es una librería de logging usada de forma transitiva en un número enorme de aplicaciones Java, lo que significaba que muchas organizaciones ni siquiera sabían con certeza en cuántos sistemas estaba presente. Los equipos que tenían inventario de software automatizado y correlacionado contra CVEs pudieron identificar sistemas afectados en horas; los que dependían de procesos manuales de auditoría de software tardaron semanas en tener siquiera un mapa completo de su exposición, ventana de tiempo que los atacantes explotaron activamente en todo el mundo mientras tanto.

Laboratorio

Comando 1:

`docker exec -it single-node-wazuh.manager-1 sed -n '/<vulnerability-detection>/,/<\vulnerability-detection>/p' /var/ossec/etc/ossec.conf`

```
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $ docker exec -it single-node-wazuh.manager-1 sed -n '/<vulnerability-
detection>/,/<\vulnerability-detection>/p' /var/ossec/etc/ossec.conf
<vulnerability-detection>
<enabled>yes</enabled>
<index-status>yes</index-status>
<feed-update-interval>60m</feed-update-interval>
</vulnerability-detection>
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $
```

Qué hace: muestra la configuración del módulo de detección de vulnerabilidades, confirmando que está habilitado y qué feeds de CVE tiene activos — sin el feed correcto para tu distribución, el módulo simplemente no tendrá contra qué correlacionar tu inventario.

Comando 2:

`docker exec -it single-node-wazuh.manager-1 bash -c "tail -f /var/ossec/logs/ossec.log | grep -i vulnerability"`

```
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $ docker exec -it single-node-wazuh.manager-1 bash -c "tail -f /var/os
sec/logs/ossec.log | grep -i vulnerability"
2026/07/21 04:34:05 wazuh-modulesd:vulnerability-scanner: INFO: Vulnerability sc
anner module started.
```

Qué hace: sigue en vivo el log principal filtrado por el submódulo de vulnerabilidades, donde puedes observar cómo descarga los feeds de CVE y comienza a correlacionarlos contra el inventario recolectado por syscollector.

Comando 3:

`curl -s -u wazuh-wui:PASSWORD -k -X GET "https://localhost:55000/vulnerability/AGENT_ID" | jq '.data.affected_items[] | {cve: .cve, package: .name, severity: .severity}' | head -30`

```
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $ curl -s -u wazuh-wui:PASSWORD -k -X GET "https://localhost:55000/vul
nerability/AGENT_ID" | jq '.data.affected_items[] | {cve: .cve, package: .name,
severity: .severity}' | head -30
jq: error (at <stdin>:0): Cannot iterate over null (null)
(jo4nlu♥parrot)-[~/zabbix-docker]
[ ✓ ] $
```

Qué hace: consulta la API REST de Wazuh para listar las vulnerabilidades (CVEs) detectadas en un agente específico, mostrando solo CVE, paquete afectado y severidad — la consulta exacta que generarías para un reporte de exposición ante un escenario tipo Log4Shell. Reemplaza PASSWORD y AGENT_ID con tus valores reales.

Tema 7: Active Response — Contención Automatizada

Concepto

Active Response es la capacidad de Wazuh de ejecutar automáticamente una acción de contención en el momento exacto en que una regla específica se dispara, sin esperar intervención humana: bloquear una IP a nivel de firewall, deshabilitar una cuenta de usuario, o matar un proceso malicioso. Es, en esencia, la automatización de la fase de **Contención** del ciclo de respuesta a incidentes que estudiaste en la Fase 1 (NIST SP 800-61).

Esta capacidad es un arma de doble filo que todo analista debe entender con respeto. La velocidad es su mayor fortaleza: cierto ransomware moderno puede cifrar una cantidad significativa de archivos en cuestión de minutos, una ventana demasiado corta para que un humano detecte, investigue y actúe manualmente a tiempo. Pero esa misma velocidad automatizada es también su mayor riesgo: una regla mal calibrada que dispare una respuesta activa de bloqueo contra tráfico legítimo puede causar una **interrupción de servicio autoinfligida** — el equivalente en seguridad de “el remedio peor que la enfermedad”.

Por esta razón, la práctica profesional recomendada es aplicar respuestas activas completamente automáticas solo a acciones de **bajo riesgo y fácilmente reversibles** (bloqueo temporal de una IP por 10 minutos tras fuerza bruta SSH, por ejemplo, usando el parámetro timeout para que el bloqueo expire solo), mientras que acciones de **alto impacto** (deshabilitar una cuenta de administrador, aislar un servidor de producción crítico) deben mantener a un humano en el circuito de decisión, incluso si la alerta que las dispara es de severidad crítica.

Caso real: la evolución del ransomware moderno hacia técnicas de cifrado cada vez más rápidas es una tendencia ampliamente documentada por múltiples firmas de respuesta a incidentes en sus reportes anuales — el tiempo entre el acceso inicial y el cifrado masivo de archivos se ha reducido significativamente en los últimos años en muchas familias de ransomware observadas en el mundo real. Esta compresión del tiempo de reacción disponible es precisamente el argumento de negocio detrás de invertir en active response bien calibrado: contra una amenaza que actúa en minutos, un playbook que depende exclusivamente de un analista humano despertando, investigando y decidiendo puede simplemente llegar demasiado tarde.

Laboratorio

Comando 1:

`docker exec -it single-node-wazuh.manager-1 sed -n '/<active-response>/,/</active-response>/p' /var/ossec/etc/ossec.conf`

```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ docker exec -it single-node-wazuh.manager-1 sed -n '/<active-response>/,/</active-response>/p' /var/ossec/etc/ossec.conf
<active-response>
  active-response options here
</active-response>
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: muestra los bloques de respuesta activa actualmente configurados en el Manager (si existen), incluyendo qué comando ejecutan (ej. firewall-drop), ante qué reglas se disparan, y con qué timeout — el punto de partida antes de definir tu propia respuesta automatizada.

Comando 2:

`docker exec -it single-node-wazuh.manager-1 cat /var/ossec/active-response/bin/firewall-drop`

```
(jo4nlu♥parrot)-[~]
[ * ] $ docker exec -it single-node-wazuh.manager-1 cat /var/ossec/active-response/bin/firewall-drop
ELF>o'@@(S@8  @s@@@@@088@8@@@lFLF 0M0M`0M``0 0M0M`0M`@TT@T@ P0td0@@@0@@0000t
dR0td0M0M`0M`XX/lib64/ld-linux-x86-64.so.2GNU CD>(0)+<?.5ACB$89@ 7%:*s,=2
/6

#&"1 !'-.;800c050r
09@0#0000000]>00@0yR000s000003%R`HRV0[0V0R`00f0nv0R`000 Rsn0
0@0librt.so.1_gmon_start_libdl.so.2libwazuhshared.socJSON_PrintUnformattedcJSO
N_ParseWithOptscJSON_IsObject_merror_exit_finicJSON_AddNumberToObjectwpopenvbase
name_exget_binary_pathcJSON_CreateArraycJSON_AddStringToObjectw_get_timestampcJS
ON_AddItemToArraycJSON_DeletcJSON_AddItemToObjectw_strtok_r_str_delimw_homedirr
mdir_excJSON_CreateStringwpclosecJSON_GetObjectItemcJSON_CreateObjectwfpopen_init
libwazuhext.socJSON_IsStringcJSON_IsNumbercJSON_GetArraySizecJSON_IsArraycJSON_G
etArrayItemlibm.so.6libpthread.so.0__errno_locationlibc.so.6_xpg_basename fflush
strcpy__strdupstrncpytimemkdstdingetpidstrtokstrtolfgetscalocstrlengetaddrinfo
omemsetstrchrdirstdoutfclosestrcatsscanfscanfstrchrfprintfsleepfreeaddrinfo
amestrcmpstrerror__libc_start_mainsprintf_free_edata__bss_start_end$ORIGIN/../../
/libGLIBC_2.2.5 usi m6usi m00` R` R'?P` P`(P`0P`8P`@P`HP`PPXP` `P`
hP`
```

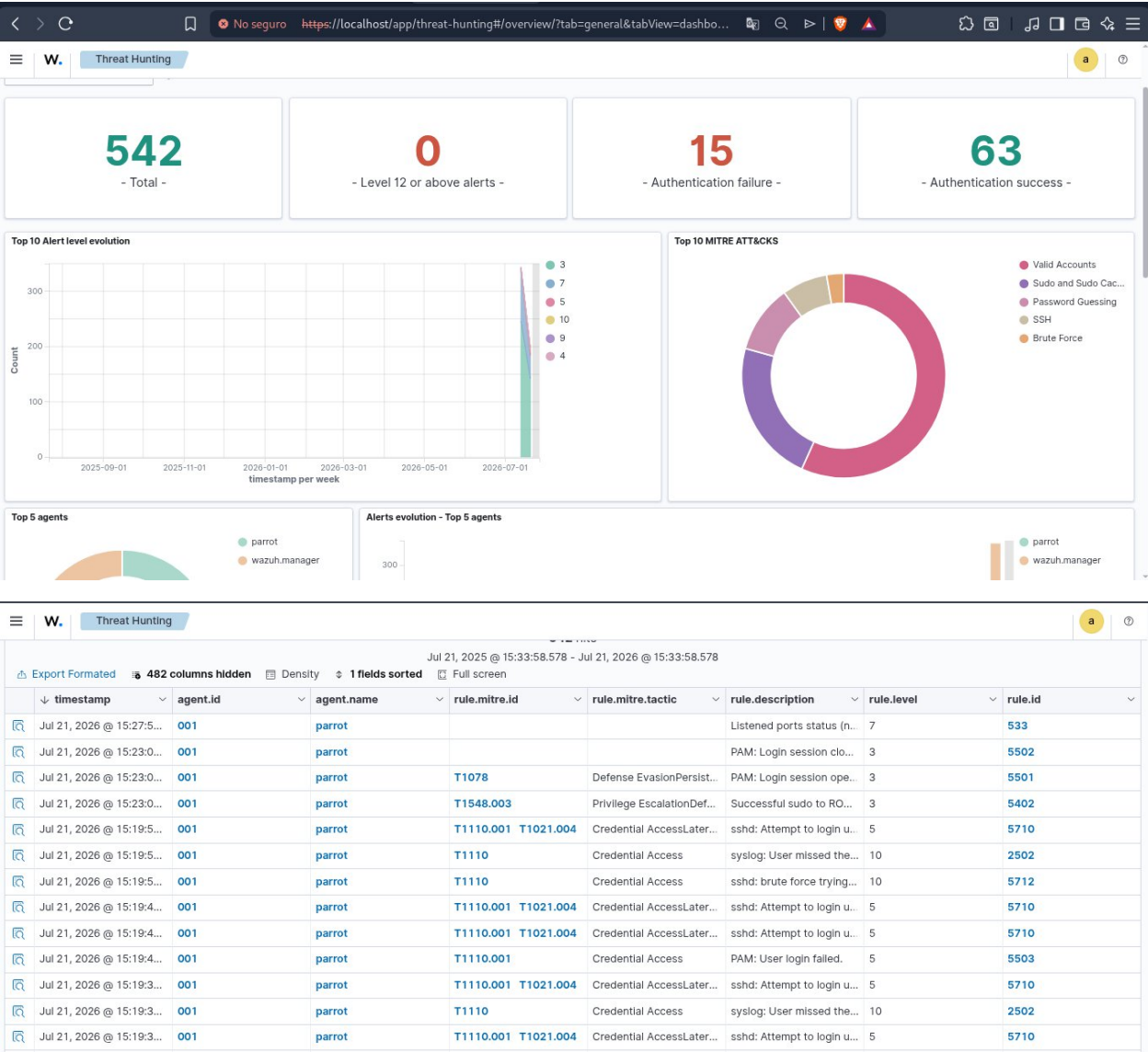
Qué hace: inspecciona el script real que Wazuh ejecuta para el bloqueo de IPs. Entender exactamente qué hace el script (típicamente invoca iptables/nft para insertar una regla DROP) es obligatorio antes de habilitarlo — una respuesta activa mal entendida puede terminar bloqueándote a ti mismo desde tu propia sesión SSH.

Comando 3:

docker exec -it single-node-wazuh.manager-1 tail -f /var/ossec/logs/active-responses.log

```
[✓] $ docker exec -it single-node-wazuh.manager-1 tail -f /var/ossec/logs/active-responses.log
Tue Jul 21 20:11:10 UTC 2026 active-response/bin/restart.sh manager
^C(jo4nlu♥parrot)-[~]
[✖] $
```

Qué hace: monitorea en vivo el log dedicado exclusivamente a respuestas activas. Cada vez que Wazuh ejecuta una acción de contención automatizada queda registrada aquí con timestamp — esencial tanto para auditar el comportamiento del sistema como para revertir manualmente una acción si resulta ser un falso positivo.



Tema 8: OpenSearch/Kibana — Consultas y Dashboards para Threat Hunting

Concepto

El **Dashboard de Wazuh** (basado en OpenSearch Dashboards, la evolución open-source de Kibana) es donde la teoría de threat hunting de la Fase 1 se vuelve trabajo diario tangible. Mientras las alertas del SIEM te dicen “esto disparó una regla que ya conocíamos”, el threat hunting va más allá: consultas exploratorias sobre datos crudos buscando comportamiento anómalo que **ninguna regla predefinida captura todavía**.

Existen dos modalidades complementarias de threat hunting. El **hunting basado en IOC** (“known-bad hunting”) busca indicadores específicos ya conocidos —un hash, una IP, un dominio— dentro de tus datos históricos, útil para determinar retroactivamente si una amenaza recién publicada ya te afectó. El **hunting basado en hipótesis** parte de una pregunta de comportamiento, no de un indicador: “muéstrame todos los procesos cuyo padre fue winword.exe” (recordando el Tema 4 de la Fase 1), sin necesidad de que exista ningún IOC conocido previamente — es la técnica que detecta amenazas verdaderamente nuevas.

Técnicamente, tanto el Dashboard como consultas directas vía terminal usan la **API de búsqueda de OpenSearch**, basada en un lenguaje de consultas JSON (Query DSL) que permite filtrar por cualquier campo indexado, combinar condiciones booleanas, y acotar por rango de tiempo. Dominar consultas directas por API —no solo la interfaz gráfica— te da una ventaja real: puedes automatizar hunts recurrentes en scripts, integrarlos en reportes programados, o ejecutarlos rápidamente desde terminal sin depender de que el navegador y el Dashboard estén disponibles.

Caso real: la campaña de **Volt Typhoon** (identificada públicamente en 2023, dirigida contra infraestructura crítica de EE.UU.) es un ejemplo documentado por CISA de un actor que deliberadamente evita malware detectable por firmas, apoyándose casi exclusivamente en herramientas legítimas ya presentes en el sistema operativo (técnica “living off the land” que ya viste en la Fase 1, Tema 6). Este tipo de actor es, por diseño, prácticamente invisible para detección basada en reglas o firmas tradicionales — solo se descubre mediante hunting basado en hipótesis: buscar patrones de uso anómalo de herramientas legítimas (comandos administrativos ejecutados desde cuentas o momentos inusuales, combinaciones raras de proceso padre-hijo) en vez de esperar que una firma de malware conocido se dispare.

Laboratorio

Comando 1:

```
curl -k -u admin:PASSWORD "https://localhost:9200/wazuh-alerts-*/_search?pretty" -H 'Content-Type: application/json' -d '{"query":{"match":{"rule.level":12}}, "size":5}'
```

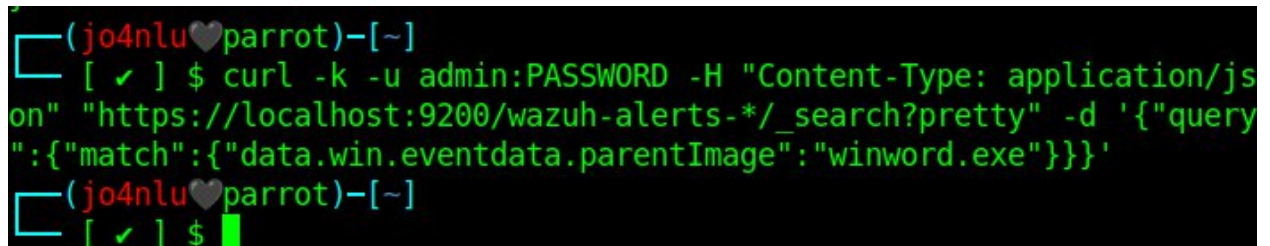


```
^C (jo4nlu♥parrot)-[~]
[ * ] $ curl -k -u admin:PASSWORD "https://localhost:9200/wazuh-alerts-*/_search?pretty" -H 'Content-Type: application/json' -d '{"query":{"match":{"rule.level":12}}, "size":5}'
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: ejecuta una consulta directa contra la API de OpenSearch buscando las 5 alertas más recientes con nivel de severidad 12 (crítico) — la misma lógica que usarías en la barra de búsqueda del Dashboard, pero desde terminal, útil para automatizar reportes o integrarla en scripts propios.

Comando 2:

```
curl -k -u admin:PASSWORD -H "Content-Type: application/json" "https://localhost:9200/wazuh-alerts-*/_search?pretty" -d '{"query":{"match":{"data.win.eventdata.parentImage":"winword.exe"}}}'
```

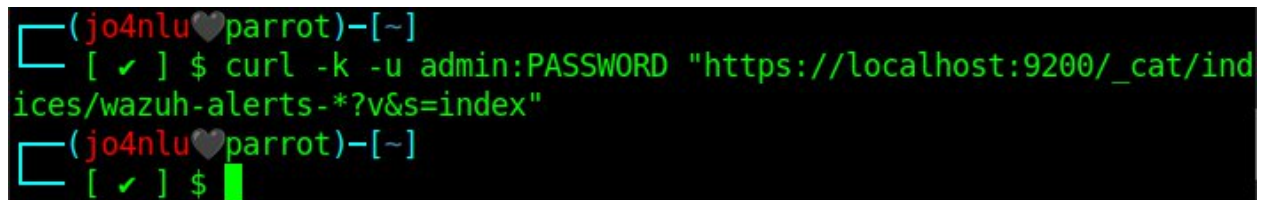


```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ curl -k -u admin:PASSWORD -H "Content-Type: application/json" "https://localhost:9200/wazuh-alerts-*/_search?pretty" -d '{"query":{"match":{"data.win.eventdata.parentImage":"winword.exe"}}}'
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: busca específicamente eventos donde el proceso padre fue winword.exe — la técnica de threat hunting basado en hipótesis del Tema 4 de la Fase 1 (cadena documento → proceso sospechoso), ahora ejecutada como consulta real contra tus datos indexados en OpenSearch.

Comando 3:

```
curl -k -u admin:PASSWORD "https://localhost:9200/_cat/indices/wazuh-alerts-*?v&s=index"
```



```
(jo4nlu♥parrot)-[~]
[ ✓ ] $ curl -k -u admin:PASSWORD "https://localhost:9200/_cat/indices/wazuh-alerts-*?v&s=index"
(jo4nlu♥parrot)-[~]
[ ✓ ] $
```

Qué hace: lista todos los índices de alertas de Wazuh con su tamaño y cantidad de documentos — te da visibilidad de cuántos datos históricos tienes disponibles para hunting, y es el primer chequeo cuando una búsqueda “no devuelve nada”: confirmar que el índice del rango de fechas que buscas realmente existe.

Cierre de Fase 2

Con estos 8 temas construiste, desde cero, un SOC funcional real sobre Wazuh: arquitectura SIEM, despliegue de infraestructura con Docker, agentes en endpoints, motor de correlación con reglas custom, monitoreo de integridad de archivos, gestión de vulnerabilidades, respuesta automatizada, y threat hunting activo con consultas directas a OpenSearch.

Próxima fase: Fase 3 — Forense digital, análisis de memoria, ingeniería inversa básica de malware, y simulacros de incidentes end-to-end (Purple Team).

Documento generado por Instructor SOC — Academia SOC Personal